DATA 260 HW9 - Farzaan Dalwai

Setup

Repository: https://github.com/farzaandalwai/data260-0686

SID4 0686 | PORT_BASE 8686 | PREFIX s0686 | SEED 686 | VERIFY_SEED 260686 | DOMAIN_ID 6 - Rental Housing Listings

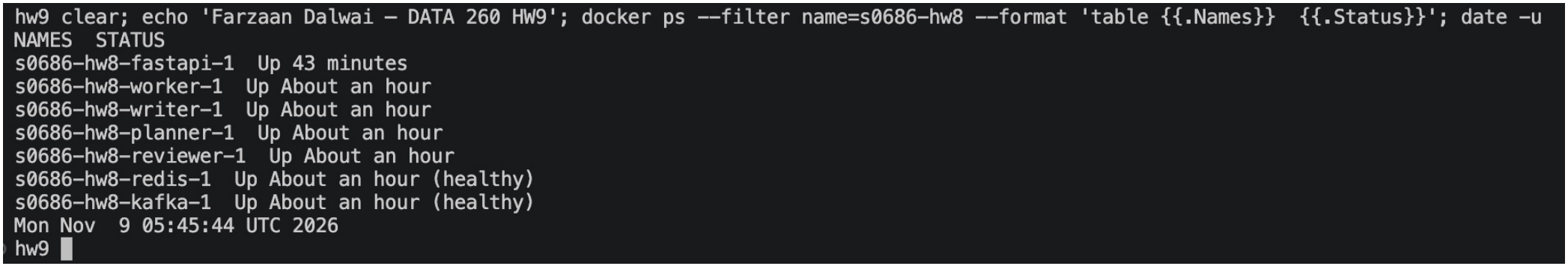
Apple M4, 24 GiB RAM, macOS 26.6.2 arm64; Python 3.11.14; Kafka 4.1.2; Redis 8.2.2. Local Ollama: llama3.2:latest for answers, nomic-embed-text:latest for 768-dimensional embeddings.

Tagged commit: hw9 (symbolic source reference; the post-tag release/submission PDF resolves the numeric hash).

The existing app, local adapter and MySQL s0686_rel are extended. Original 5,000 records were preserved; private backups preceded the schema additions. GitHub read access was observed for Sbnikitha and supriyaselvanganesan, but exact collaborator membership was not confirmed and no access changes were made.

compose.hw09.yml, lines 1-9 (excerpt; surrounding lines omitted)

```
services:
  redis:
    image: redis:8.2.2
    ports:
      - "127.0.0.1:6386:6379"
    healthcheck:
      test: ["CMD", "redis-cli", "PING"]
      interval: 3s
      timeout: 3s
```



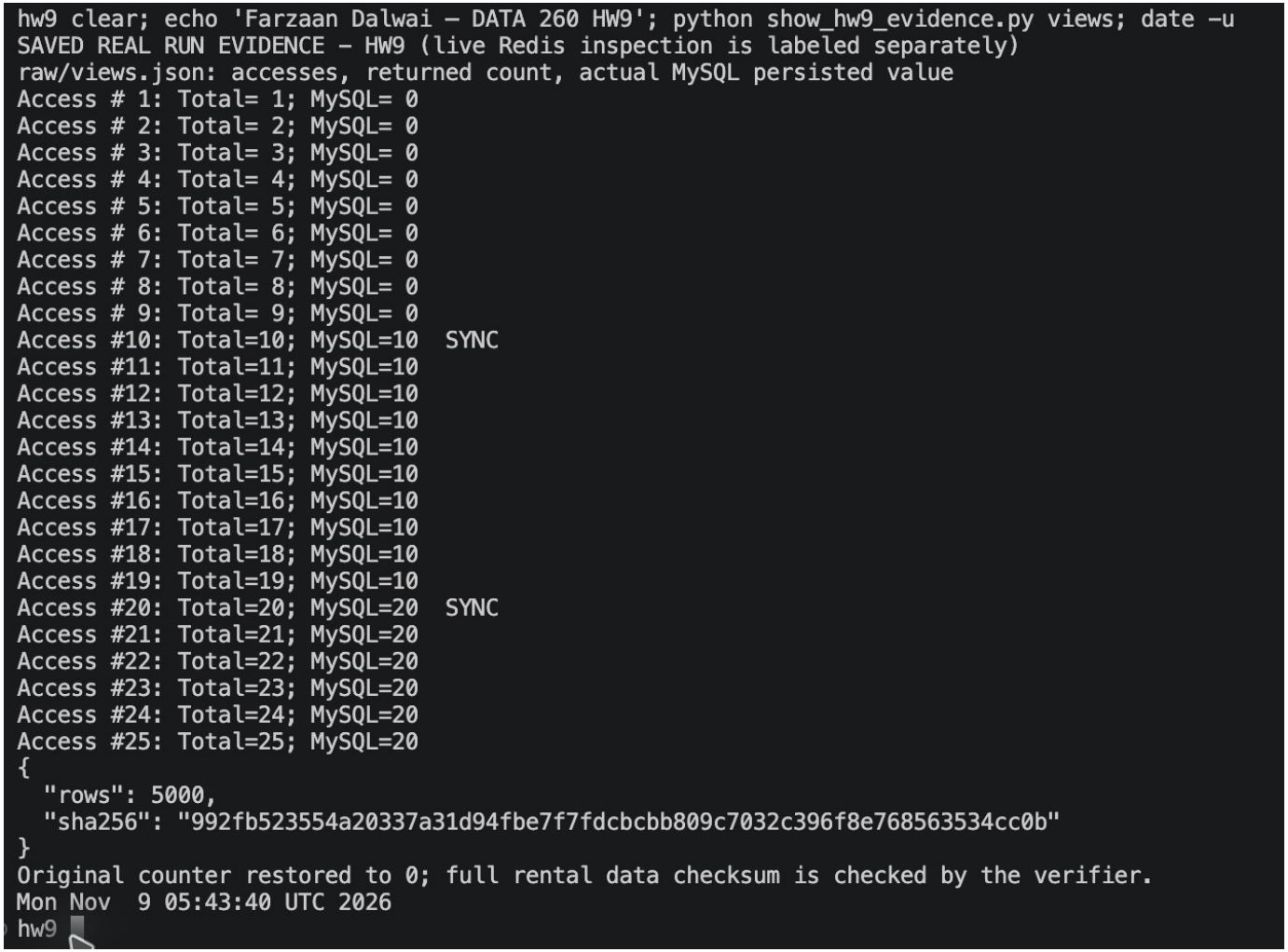
Part 1 - Views

25 accesses

hw9_cache.py, lines 81-86 (excerpt; surrounding lines omitted)

```
        count = self.redis_client.incr(key)
        if count % 10 == 0:
            with db_engine.begin() as connection:
                connection.execute(text("UPDATE rental_listings SET view_count=GREATEST(view_count,:count) WHERE id=:id"), {"count": count, "id":
    listing_id})
            log("views_synced", listing_id=listing_id, count=count, message="Synced views to database")
        return count
```

Redis INCR returns every new count; MySQL syncs at 10 and 20, and the demo restores its original counter afterward.



Part 2 - Vector cache

Redis cosine index

hw9_cache.py, lines 89-93 (excerpt; surrounding lines omitted)

```
def embed(query: str) -> list[float]:
    response = requests.post(os.getenv("HW9_EMBED_URL", "http://127.0.0.1:11434/api/embed"),
                             json={"model": "nomic-embed-text:latest", "input": query}, timeout=120)
    response.raise_for_status()
    vector = response.json()["embeddings"][0]
```

hw9_cache.py, lines 120-126 (excerpt; surrounding lines omitted)

```
        try:
            self.client.execute_command("FT.CREATE", INDEX, "ON", "HASH", "PREFIX", 1, PREFIX + ":entry:",
                                        "SCHEMA", "namespace", "TAG", "version", "NUMERIC", "embedding", "VECTOR", "FLAT", 6,
                                        "TYPE", "FLOAT32", "DIM", 768, "DISTANCE_METRIC", "COSINE")
        except redis.ResponseError as error:
            if "Index already exists" not in str(error):
                raise
```

hw9_cache.py, lines 128-134 (excerpt; surrounding lines omitted)

```
    def put(self, query: str, vector: list[float], result: dict, version: int, listing_id: int, field: str):
        key = f"{PREFIX}:entry:{self.namespace}:{uuid4().hex}"
        self.client.hset(key, mapping={"namespace": self.namespace, "version": version, "query": query,
                                       "embedding": array("f", vector).tobytes(), "answer": result["answer"],
                                       "correlation_id": result["correlation_id"], "listing_id": listing_id, "field": field})
        self.client.expire(key, 3600)
        return key
```

The Redis hash stores the query, FLOAT32 embedding, final answer and version; all keys/indexes begin with s0686 and entries expire after one hour.

```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py index; date -u
SAVED REAL RUN EVIDENCE - HW9 (live Redis inspection is labeled separately)
LIVE Redis FT.INFO:
index_name: b's0686:hw9:semantic_index'
index_definition: [b'key_type', b'HASH', b'prefixes', [b's0686:hw9:entry:'], b'default_score', b'1', b'indexes_all', b'false']
num_docs: 9
Selected actual FT.INFO field attributes:
  identifier : b'namespace'
  type : b'TAG'
  identifier : b'version'
  type : b'NUMERIC'
  identifier : b'embedding'
  type : b'VECTOR'
  algorithm : b'FLAT'
  data_type : b'FLOAT32'
  dim : 768
  distance_metric : b'COSINE'
Keys (examples): ['s0686:hw9:entry:live:5840911193d94e298b8d9471a92918c1', 's0686:hw9:entry:live:d0666c01846349cf8064cc172c3bcbf5', 's0686:hw9:entry:live:1ad1a17327ec48c68a2
1c457b1879e6e', 's0686:hw9:data_version', 's0686:hw9:entry:live:2a858ae916b44db39f89b7fd3255ba18']
Mon Nov  9 05:44:04 UTC 2026
hw9
```

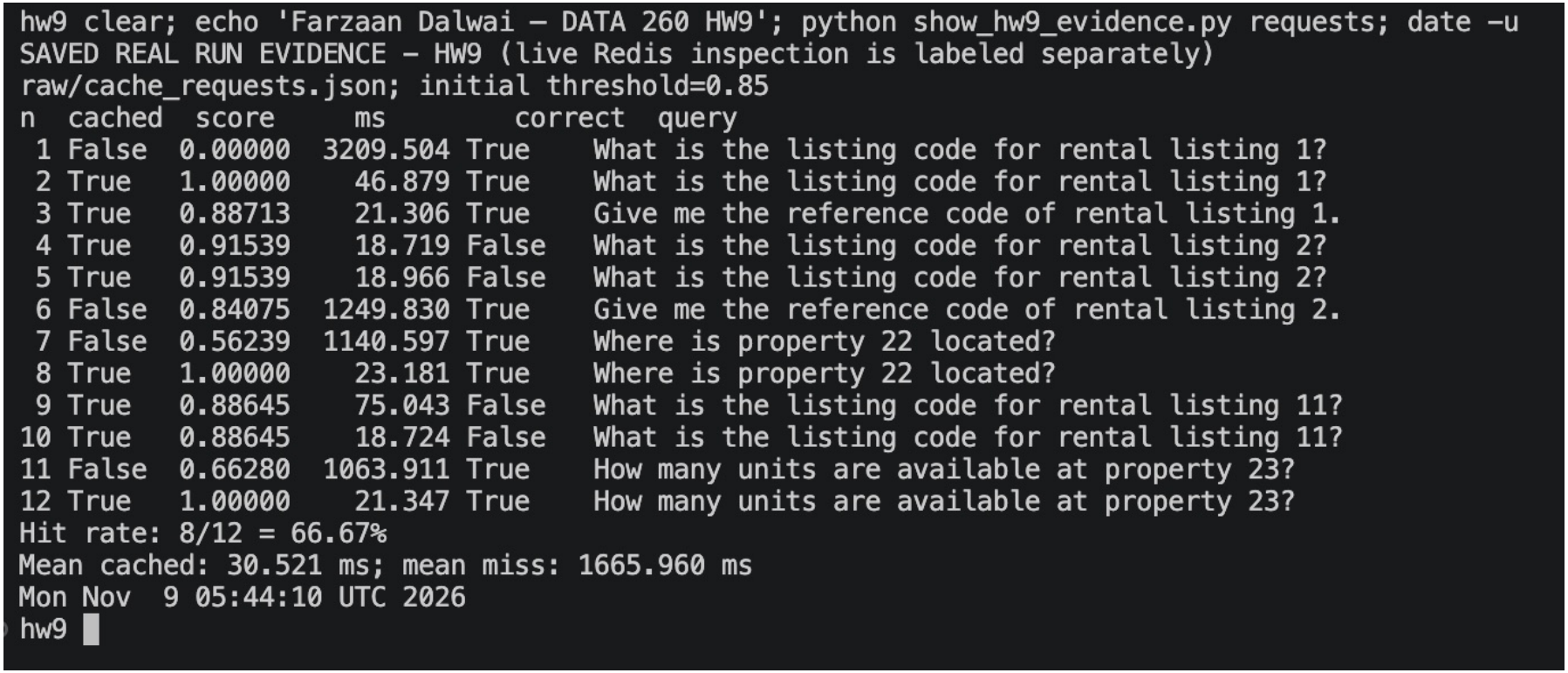
Part 2 - Requests

Initial threshold 0.85

hw9_cache.py, lines 144-158 (excerpt; surrounding lines omitted)

```
        similarity = max(-1.0, min(1.0, 1 - float(match[b"distance"]))) if match else 0.0
        hit = bool(match) and similarity >= self.threshold
        if hit:
            answer, cid = match[b"answer"].decode(), match[b"correlation_id"].decode()
        else:
            result = pipeline_answer(query, listing_facts(listing_id, field))
            answer, cid = result["answer"], result["correlation_id"]
            if store_miss:
                self.put(query, vector, result, version, listing_id, field)
        response = {"query": query, "listing_id": listing_id, "field": field, "answer": answer,
                    "is_cached": hit, "similarity_score": round(similarity, 8), "data_version": version,
                    "latency_ms": round((time.perf_counter() - started) * 1000, 3), "correlation_id": cid,
                    "matched_query": match.get(b"query", b"").decode(), "threshold": self.threshold}
        log("cache_hit" if hit else "cache_miss", namespace=self.namespace, **response)
        return response
```

The 12 authenticated requests produced 8 hits (66.67%); mean hit/miss latency was 30.521/1665.960 ms, with four wrong returned answers. Handler timing includes embedding, version lookup, Redis and miss work; HTTP authentication is excluded.



```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py requests; date -u
SAVED REAL RUN EVIDENCE - HW9 (live Redis inspection is labeled separately)
raw/cache_requests.json; initial threshold=0.85
n  cached  score     ms        correct  query
 1 False  0.00000  3209.504 True    What is the listing code for rental listing 1?
 2 True   1.00000    46.879 True    What is the listing code for rental listing 1?
 3 True   0.88713    21.306 True    Give me the reference code of rental listing 1.
 4 True   0.91539    18.719 False   What is the listing code for rental listing 2?
 5 True   0.91539    18.966 False   What is the listing code for rental listing 2?
 6 False  0.84075  1249.830 True    Give me the reference code of rental listing 2.
 7 False  0.56239  1140.597 True    Where is property 22 located?
 8 True   1.00000    23.181 True    Where is property 22 located?
 9 True   0.88645    75.043 False   What is the listing code for rental listing 11?
10 True   0.88645    18.724 False   What is the listing code for rental listing 11?
11 False  0.66280  1063.911 True    How many units are available at property 23?
12 True   1.00000    21.347 True    How many units are available at property 23?
Hit rate: 8/12 = 66.67%
Mean cached: 30.521 ms; mean miss: 1665.960 ms
Mon Nov  9 05:44:10 UTC 2026
hw9
```

Part 2 - Pipeline

Real miss and repeated hit

routers/hw9_cache.py, lines 21-25 (excerpt; surrounding lines omitted)

```
@router.post("/ask")
def ask(question: Question, user=Depends(current_user)):
    try:
        listing_facts(question.listing_id, question.field)
        return SemanticCache(threshold=question.threshold if question.threshold is not None else shipped_threshold()).answer(question.query,
    question.listing_id, question.field)
```

hw9_cache.py, lines 99-111 (excerpt; surrounding lines omitted)

```
def pipeline_answer(query: str, facts: dict) -> dict:
    cid = str(uuid4())
    reply = consumer("s0686_hw9_" + cid, FINAL)
    try:
        message = {"correlation_id": cid, "question": query, "facts": facts, "use_model": True,
                   "response_json": True, "entity_key": "rental-" + str(facts["listing_id"])}
        metadata = send(producer(), INBOX, message, key=message["entity_key"])
        log("pipeline_ingress", correlation_id=cid, query=query, **metadata)
        result = wait_for(reply, cid, timeout=180)
        log("pipeline_final", correlation_id=cid, status=result["status"], answer=result["answer"], model=result.get("model"))
        return result
    finally:
        reply.close()
```

A miss traverses inbox -> Planner -> tasks -> Writer/local adapter -> drafts -> Reviewer -> final; the repeated hit returns the same final answer/correlation ID without publishing new Kafka work.

```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py journey; date -u
SAVED REAL RUN EVIDENCE — HW9 (live Redis inspection is labeled separately)
Miss query: What is the listing code for rental listing 1? correlation_id: 88f30ac2-e071-446e-bf12-fc5762373cf3
2026-11-09T05:02:47.839945+00:00 stage_receive planner s0686_inbox partition 0 offset 103
2026-11-09T05:02:47.843183+00:00 stage_send planner s0686_tasks partition 0 offset 154
2026-11-09T05:02:47.843777+00:00 stage_receive writer s0686_tasks partition 0 offset 154
2026-11-09T05:02:50.216362+00:00 stage_send writer s0686_drafts partition 0 offset 204
2026-11-09T05:02:50.219717+00:00 stage_receive reviewer s0686_drafts partition 0 offset 204
2026-11-09T05:02:50.227810+00:00 stage_send reviewer s0686_final partition 0 offset 204
Actual final answer: {"listing_id": 1, "listing_code": "RH-0686-00001"}
Repeated request: cached= True same final correlation_id= 88f30ac2-e071-446e-bf12-fc5762373cf3
Writer model: llama3.2:latest; cache-hit no-publish behavior is separately smoke-tested.
Mon Nov  9 05:44:18 UTC 2026
hw9
```

Part 3.1 - Frozen cases

40 labels before the sweep

scripts/hw9_freeze_cases.py, lines 22-32 (excerpt; surrounding lines omitted)

```
        cases.extend([
            {"case_id": f"exact_{index + 1:02}", "category": "exact", "should_hit": True,
             "listing_id": seed_id, "field": "listing_code", "query": base},
            {"case_id": f"paraphrase_{index + 1:02}", "category": "paraphrase", "should_hit": True,
             "listing_id": seed_id, "field": "listing_code", "query": f"Give me the reference code of rental listing {seed_id}."},
            {"case_id": f"similar_{index + 1:02}", "category": "similar_different", "should_hit": False,
             "listing_id": other_id, "field": "listing_code", "query": f"What is the listing code for rental listing {other_id}?"},
            {"case_id": f"unrelated_{index + 1:02}", "category": "unrelated", "should_hit": False,
             "listing_id": unrelated_id, "field": "location" if index % 2 == 0 else "available_units",
             "query": f"Where is property {unrelated_id} located?" if index % 2 == 0 else f"How many units are available at property
    {unrelated_id}?"},
        ])
```

Commit 0cdc5328e78881f95f5756247ba0ee95c55162a6 fixed ten cases per category before measurements. Labels stayed unchanged; no labeling error was found. Each threshold starts from the same ten actual warm-up answers; benchmark misses do not add entries.

```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py labels; date -u
SAVED REAL RUN EVIDENCE — HW9 (live Redis inspection is labeled separately)
0cdc5328e78881f95f5756247ba0ee95c55162a6
2026-11-08T20:13:51-08:00
Freeze HW9 cache labels, input splits and cost plan before measurements

SHA256: 777de55ec8aba136ebc8fc3f5a8a57a9a20351121011ff636af68ec6e2ea70a3
VERIFY_SEED=260686; 40 cases
exact count= 10 should_hit= True
  Example: What is the listing code for rental listing 6?
paraphrase count= 10 should_hit= True
  Example: Give me the reference code of rental listing 7.
similar_different count= 10 should_hit= False
  Example: What is the listing code for rental listing 18?
unrelated count= 10 should_hit= False
  Example: Where is property 28 located?
Same ten warm-up answers at every threshold; misses are not added to the frozen sweep cache.
Mon Nov  9 05:44:24 UTC 2026
hw9
```

Parts 3.2 and 3.4 - Thresholds

Measured trade-off

Threshold	Hit rate	False-hit rate	Mean latency (ms)	Total cost
0.75	75.0%	25.0%	285.023	210
0.80	75.0%	25.0%	283.669	210
0.85	75.0%	25.0%	282.772	210
0.90	55.0%	7.5%	523.583	78
0.95	25.0%	0.0%	865.898	30

Costs were fixed before the sweep: miss=1 for extra local inference work; false hit=20 for a wrong housing answer and support correction. These are normalized assumptions, not dollars. Total=misses+20*false_hits; false-hit rate=wrong cached answers/40. Ship 0.95: lowest cost (30), with zero false hits in this set; lower hit rate and higher mean latency are the trade-off.

run_hw9_cache.py, lines 108-114 (excerpt; surrounding lines omitted)

```
        hits = sum(row["is_cached"] for row in rows)
        false_hits = sum(row["false_hit"] for row in rows)
        result = {"threshold": threshold, "cases": len(rows), "hits": hits, "misses": len(rows) - hits,
                  "false_hits": false_hits, "hit_rate": hits / len(rows), "false_hit_rate": false_hits / len(rows),
                  "false_hit_given_hit": false_hits / hits if hits else 0,
                  "mean_latency_ms": round(sum(row["latency_ms"] for row in rows) / len(rows), 3),
                  "total_cost": len(rows) - hits + 20 * false_hits}
```

```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py thresholds; date -u
SAVED REAL RUN EVIDENCE — HW9 (live Redis inspection is labeled separately)
threshold hits/40 false_hits/40 mean_ms cost_units(misses + 20*false_hits)
0.75     30/40   10/40         285.023    210
0.80     30/40   10/40         283.669    210
0.85     30/40   10/40         282.772    210
0.90     22/40    3/40         523.583    78
0.95     10/40    0/40         865.898    30
Shipped threshold: 0.95 Minimum observed cost; higher threshold breaks ties
Assumed costs: miss=1, wrong cached answer=20 normalized units; not monetary measurements.
Mon Nov  9 05:44:30 UTC 2026
hw9
```

Part 3.3 - False hit

Observed confusion

hw9_cache.py, lines 140-147 (excerpt; surrounding lines omitted)

```
        search = f"(@namespace:{{{self.namespace}}} @version:[{version} {version}])=>[KNN 1 @embedding $vector AS distance]"
        found = self.client.execute_command("FT.SEARCH", INDEX, search, "PARAMS", 2, "vector", array("f", vector).tobytes(),
                                            "SORTBY", "distance", "RETURN", 7, "query", "answer", "version", "correlation_id", "listing_id", "field",
     "distance", "DIALECT", 2)
        match = dict(zip(found[2][::2], found[2][1::2])) if found[0] else {}
        similarity = max(-1.0, min(1.0, 1 - float(match[b"distance"]))) if match else 0.0
        hit = bool(match) and similarity >= self.threshold
        if hit:
            answer, cid = match[b"answer"].decode(), match[b"correlation_id"].decode()
```

Likely confusion: repeated listing-code wording outweighed the changed numeric record ID, returning another listing's answer; 0.95 is not a universal correctness guarantee.

```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py falsehit; date -u
SAVED REAL RUN EVIDENCE - HW9 (live Redis inspection is labeled separately)
Observed false hit: case similar_08 threshold 0.75
Cached query: What is the listing code for rental listing 8?
Incoming query: What is the listing code for rental listing 18?
Cosine similarity: 0.90014946
Returned cached answer: {"listing_id": 8, "listing_code": "RH-0686-00008"}
Correct MySQL answer: {"listing_id": 18, "listing_code": "RH-0686-00018"}
is_cached: True answer_correct: False
Likely confusion: same wording about listing codes; the changed numeric record ID is not separated enough by the embedding.
Mon Nov  9 05:44:37 UTC 2026
hw9
```

Part 3.5 - Stale answer

Version invalidation

hw9_cache.py, lines 52-61 (excerpt; surrounding lines omitted)

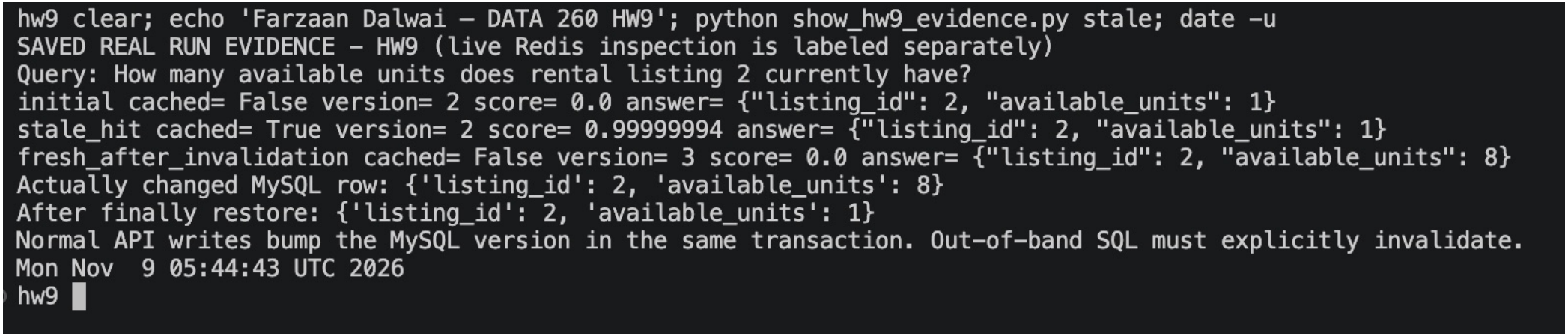
```
def bump_data_version(db) -> None:
    db.execute(text("UPDATE s0686_cache_version SET version=version+1 WHERE id=1"))


def invalidate_data() -> int:
    with db_engine.begin() as connection:
        bump_data_version(connection)
        version = connection.execute(text("SELECT version FROM s0686_cache_version WHERE id=1")).scalar_one()
    log("data_invalidated", version=version)
    return version
```

hw9_cache.py, lines 137-142 (excerpt; surrounding lines omitted)

```
        started = time.perf_counter()
        vector, version = embed(query), data_version()
        self.client.set(PREFIX + ":data_version", version)
        search = f"(@namespace:{{{self.namespace}}} @version:[{version} {version}])=>[KNN 1 @embedding $vector AS distance]"
        found = self.client.execute_command("FT.SEARCH", INDEX, search, "PARAMS", 2, "vector", array("f", vector).tobytes(),
                                            "SORTBY", "distance", "RETURN", 7, "query", "answer", "version", "correlation_id", "listing_id", "field",
     "distance", "DIALECT", 2)
```

Changing listing 2 from 1 to 8 units left the old cached answer at version 2; version 3 forced a fresh answer of 8. The original row/timestamp was restored and version advanced to 4. Normal API writes bump the version in the same SQL transaction; out-of-band SQL needs explicit invalidation. A global version invalidates unrelated answers and adds a SQL read on hits; Redis loss can lose up to nine unsynced views.



Part 4 - Datasets

Facts separate from inputs

scripts/hw9_freeze_cases.py, lines 37-45 (excerpt; surrounding lines omitted)

```
    for name, start, fields in (("training", 30, [["available_units"], ["available_units"], ["available_units"], ["location"]]),
                                ("validation", 34, [["available_units"], ["available_units", "location"], ["location"], ["property_title"],
    ["listing_code"], ["pet_policy"]]),
                                ("test", 40, [["available_units"], ["location"], ["listing_code"], ["property_title"], ["available_units",
    "location"], ["pet_policy"], ["available_units"], ["location", "listing_code"]])):
        inputs = []
        for index, requested in enumerate(fields):
            listing_id = rows[start + index]["listing_id"]
            question = f"For rental listing {listing_id}, report {', '.join(requested)} from the supplied record; use null when not recorded."
            inputs.append({"question": question, "listing_id": listing_id, "fields": requested})
        Path(name + "_data.json").write_text(json.dumps(inputs, indent=2) + "\n")
```

hw9_reference.json contains 70 real rental facts; the disjoint input-only files contain 4 training, 6 validation and 8 held-out questions. The same test bytes were first loaded after lesson selection and used unchanged across all four variants.

```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py datasets; date -u
SAVED REAL RUN EVIDENCE — HW9 (live Redis inspection is labeled separately)
Separate reference snapshot: 70 real rental records
{
  "listing_id": 32,
  "property_title": "Rental Listing 0032",
  "location": "San Jose, CA",
  "listing_code": "RH-0686-00032",
  "available_units": 1
}
training_data.json count= 4 input keys= ['question', 'listing_id', 'fields']
  Input example: {"question": "For rental listing 32, report available_units from the supplied record; use null when not recorded.", "listing_id": 32, "fields": ["available_
units"]}
  SHA256: a32012f5650d105f09ad52a69688a66cc4c11ff3cc91518894b012e268d30387
validation_data.json count= 6 input keys= ['question', 'listing_id', 'fields']
  Input example: {"question": "For rental listing 36, report available_units from the supplied record; use null when not recorded.", "listing_id": 36, "fields": ["available_
units"]}
  SHA256: a9c7d4e84cc88466cdbd09a6112ac527b9da8e9ef6ec943dcd58e88571a4120d
test_data.json count= 8 input keys= ['question', 'listing_id', 'fields']
  Input example: {"question": "For rental listing 42, report available_units from the supplied record; use null when not recorded.", "listing_id": 42, "fields": ["available_
units"]}
  SHA256: 55febb6b3dcee689d0007b620eda2d9f411124088f4c9f4d110e02ce3103ed19
Mon Nov  9 05:44:49 UTC 2026
hw9
```

Part 4.1 - Repair

Answering Agent and Evaluator Agent

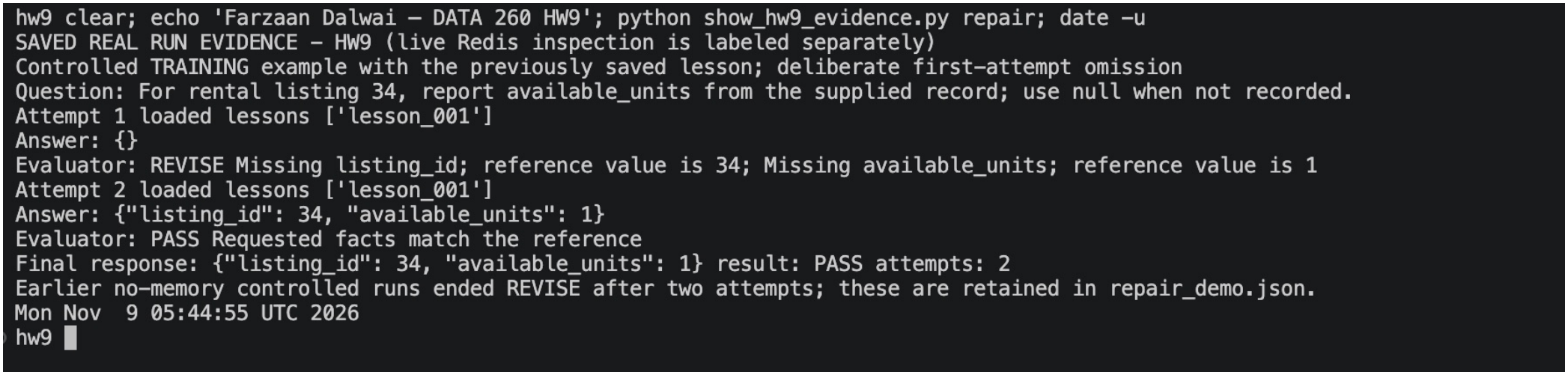
hw9_agents.py, lines 23-28 (excerpt; surrounding lines omitted)

```
        prompt = {"question": item["question"], "reference_record": record,
                  "requested_fields": ["listing_id", *item["fields"]],
                  "lessons": [lesson["correction"] for lesson in relevant], "evaluator_feedback": feedback}
        result = complete([{"role": "system", "content": "Answer from the supplied rental reference only. Return a JSON object with every requested
    field. Use null if a fact is not recorded; do not invent facts."},
                           {"role": "user", "content": json.dumps(prompt)}], temperature=0.0, json_output=True, num_predict=160)
        answer = json.loads(result["message"])
```

hw9_agents.py, lines 46-54 (excerpt; surrounding lines omitted)

```
            for field, value in expected.items():
                if field not in answer:
                    errors.append(f"Missing {field}; reference value is {json.dumps(value)}")
                elif answer[field] != value or type(answer[field]) is not type(value):
                    errors.append(f"Incorrect {field}; reference value is {json.dumps(value)}")
            for field, value in answer.items():
                if field not in expected and (field not in record or record[field] != value):
                    errors.append(f"Unsupported extra field {field}")
        return {"result": "REVISE" if errors else "PASS", "feedback": "; ".join(errors) if errors else "Requested facts match the reference"}
```

The local Answering Agent receives reference facts, lessons and feedback; the deterministic Evaluator checks values/types and unsupported fields. This training-only demo deliberately removes available_units on attempt 1, then uses evaluator feedback and the saved lesson to pass on attempt 2. Earlier no-memory controls remained REVISE at the same limit and are retained in raw/repair_demo.json.



Part 4.2 - Lessons

Save, reload and reuse

hw9_agents.py, lines 15-22 (excerpt; surrounding lines omitted)

```
    def answer(self, item: dict, feedback=None, controlled_omission=False) -> dict:
        record = self.reference[str(item["listing_id"])]
        relevant = [lesson for lesson in self.lessons
                    if ("available_units" in lesson["failure_pattern"] and "available_units" in item["fields"])
                    or ("unknown" in lesson["failure_pattern"] and any(field not in record for field in item["fields"]))]
        for lesson in relevant:
            lesson["use_count"] += 1
        log("lessons_loaded", question=item["question"], lesson_ids=[lesson["lesson_id"] for lesson in relevant])
```

run_hw9_agents.py, lines 21-26 (excerpt; surrounding lines omitted)

```
    lessons = [lesson("lesson_001", "Repeated omission of available_units",
                      "When available_units is requested, include its current reference value, including zero; do not omit it.")]
    Path("lessons.json").write_text(json.dumps(lessons, indent=2) + "\n")
    reloaded = json.loads(Path("lessons.json").read_text())
    reuse = run_agents(training[2], reference, reloaded)
    save("lesson_reuse.json", reuse)
```

Repeated training omissions created lesson_001 with lesson_id, failure_pattern, correction, created_at and use_count; the later related question reloads it from lessons.json before answering.

```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py lessons; date -u
SAVED REAL RUN EVIDENCE - HW9 (live Redis inspection is labeled separately)
[
  {
    "lesson_id": "lesson_001",
    "failure_pattern": "Repeated omission of available_units",
    "correction": "When available_units is requested, include its current reference value, including zero; do not omit it.",
    "created_at": "2026-11-09T04:52:39.871970+00:00",
    "use_count": 2
  }
]
Later related training question: For rental listing 34, report available_units from the supplied record; use null when not recorded.
attempt 1 loaded ['lesson_001'] answer {'available_units': 1} evaluation REVISE
attempt 2 loaded ['lesson_001'] answer {'available_units': 1} evaluation REVISE
Mon Nov  9 05:45:03 UTC 2026
hw9
```

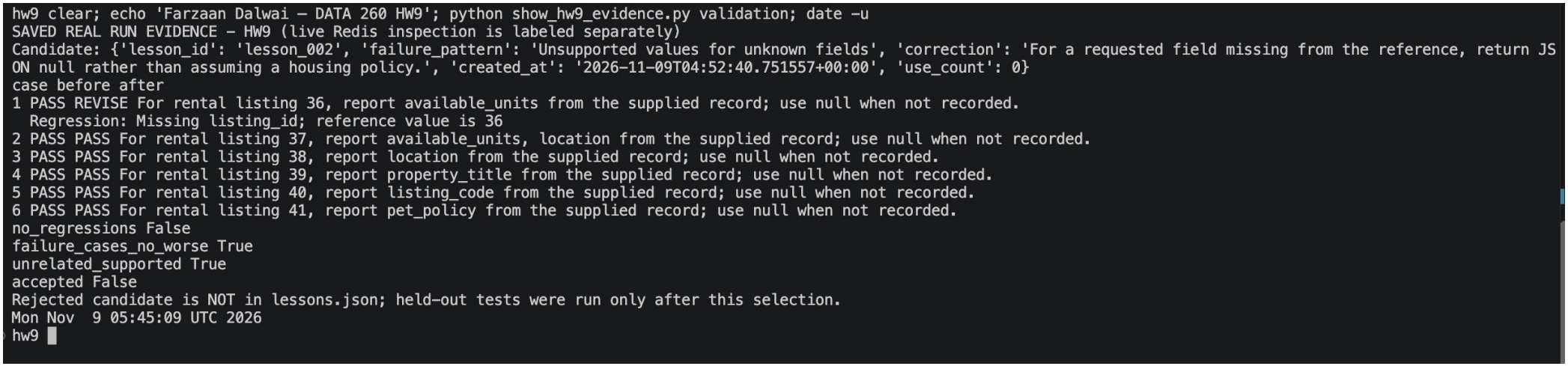
Part 4.3 - Validation

Reject the regression

hw9_agents.py, lines 87-95 (excerpt; surrounding lines omitted)

```
    old = [row["final_evaluation"] == "PASS" for row in before]
    new = [row["final_evaluation"] == "PASS" for row in after]
    no_regressions = all(not correct or new[index] for index, correct in enumerate(old))
    failures_no_worse = sum(new[index] for index, correct in enumerate(old) if not correct) >= 0
    unrelated_supported = all(new[index] for index, item in enumerate(inputs) if "available_units" not in item["fields"])
    return {"before": before, "after": after, "no_regressions": no_regressions,
            "failure_cases_no_worse": failures_no_worse, "unrelated_supported": unrelated_supported,
            "accepted": no_regressions and failures_no_worse and unrelated_supported,
            "candidate": proposed, "lessons_after": after_lessons}
```

The extra unknown-field lesson changed validation from 6/6 to 5/6: a previously correct answer regressed, so the candidate was rejected and removed. There were no failing baseline validation cases to worsen; unrelated questions stayed supported. The four held-out variants were then evaluated independently.



Part 4.4 - Held-out results

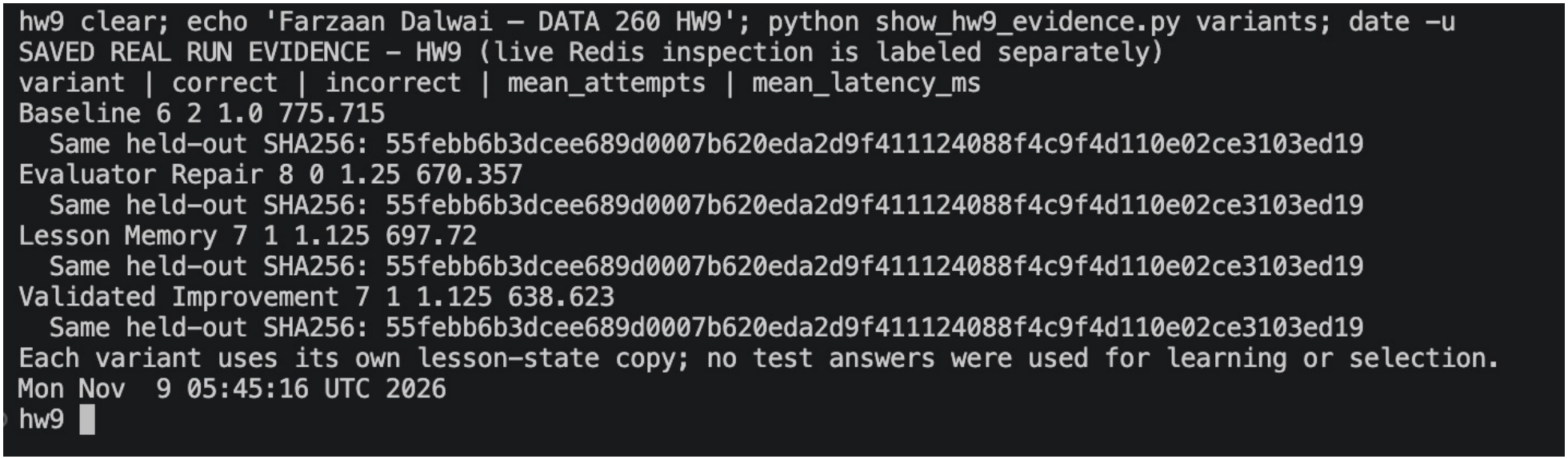
Four independent variants

Variant	Correct	Incorrect	Average attempts	Average latency (ms)
Baseline	6	2	1.000	775.715
Evaluator Repair	8	0	1.250	670.357
Lesson Memory	7	1	1.125	697.720
Validated Improvement	7	1	1.125	638.623

hw9_agents.py, lines 61-69 (excerpt; surrounding lines omitted)

```
    for attempt in range(1, 3 if repair else 2):
        generated = answering.answer(item, feedback, controlled_omission=controlled and attempt == 1)
        evaluation = evaluator.evaluate(item, generated["answer"])
        row = {"attempt": attempt, **generated, "evaluation": evaluation}
        attempts.append(row)
        log("agent_attempt", question=item["question"], **row)
        if evaluation["result"] == "PASS":
            break
        feedback = evaluation["feedback"]
```

Agents exchange JSON answers and concise feedback; PASS stops immediately, otherwise repair stops after attempt 2. Each variant has its own lesson copy. Validated Improvement kept only the prior lesson because the candidate failed validation. Limitation: small test set and local model/runtime variation; memory did not outperform Evaluator Repair, and timings do not prove a general speedup.



Verification

Objective checks

verify_hw09.py, lines 36-39 (excerpt; surrounding lines omitted)

```
    def check(name, actual, expected=True):
        passed = actual == expected
        checks.append({"name": name, "passed": passed, "actual": actual, "expected": expected})
        print(f"{'PASS' if passed else 'FAIL'} {name}: {actual}", flush=True)
```

52/52 checks passed, covering counter sync/restoration, vector cache and Kafka bypass, all 200 sweep cases/costs, stale/fresh answers, datasets, lessons, validation and four held-out totals. Existing HW8 CREATE/GET and local-model path, JWT, MySQL, MongoDB/MCP checks passed; React build/lint also passed. All original fields of all 5,000 SQL rows retain SHA256 992fb523554a20337a31d94fbe7f7fdcbcbb809c7032c396f8e768563534cc0b.

```
hw9 clear; echo 'Farzaan Dalwai — DATA 260 HW9'; python show_hw9_evidence.py checks | tail -n 18; date -u
PASS validation gate preserves correct answers : True
PASS candidate kept only if accepted : False
PASS four independent variants : ['Baseline', 'Evaluator Repair', 'Lesson Memory', 'Validated Improvement']
PASS Baseline held-out accounting : True
PASS Evaluator Repair held-out accounting : True
PASS Lesson Memory held-out accounting : True
PASS Validated Improvement held-out accounting : True
PASS all reference facts still match MySQL : True
PASS existing HW8 CREATE/GET still works : True
PASS existing HW7 auth still requires token : 401
PASS HW9 ingress preserves JWT protection : 401
PASS FastAPI existing listing routes remain : True
PASS existing HW8 freeform local-model path : True
PASS existing authenticated MySQL endpoint : True
PASS existing question CLI uses authenticated cache ingress : True
PASS existing MongoDB document endpoint : True
PASS existing MCP rental search : True
RESULT: 52 / 52
Mon Nov  9 05:45:26 UTC 2026
hw9
```

AI use

Four answers

1. I used ChatGPT/Codex to read the HW9 instructions, plan the experiments, implement and debug the code, create the datasets, run the checks, and operate my Mac for screenshots. I supplied the assignment, repository, and constraints. The assistant performed most of the technical work; I am responsible for reviewing the submission.

2. The initial Redis parsing assumed RESP2, but the installed client returned RESP3. This produced a KeyError that the first HTTP wrapper incorrectly presented as a missing listing. The local model also initially omitted listing_id from JSON answers. An initial console tee ran before its output directory existed; the actual setup JSON and private backups were retained.

3. These problems were detected from real HTTP responses, the direct traceback, and saved model output. After fixing the protocol and output schema, the requests were rerun. The 40 labels were committed before measurements and remained unchanged. Database checksums, Redis metadata, actual Kafka correlation IDs, per-case records, validation results, and the unchanged held-out test hash were checked against the recorded outputs.

4. The Redis client now explicitly uses RESP2, and missing-listing errors have their own exception. The existing local adapter accepts a JSON schema so the cache path requires all response keys without forcing their values. The cache uses a transactional data version and a measured threshold choice. Validation rejected the proposed extra lesson because it regressed one case. Earlier no-memory controlled repairs still failed at the two-attempt limit; a later training-only example with the saved lesson repaired successfully. The report keeps those outcomes and does not claim lessons always improve answers.